



Chapitre 1 : Introduction aux concepts d'algorithmique

EC : Algorithmique

Dr Ousmane KHOUMA

Email : ousmane.khouma@uam.edu.sn

Licence 1

ES2A/ESU2A/HEEG

Année académique 2020 - 2021

Objectifs spécifiques



- Être capable d'analyser un problème ;
- Pouvoir déterminer la structure d'un programme algorithmique ;
- Maîtriser les entrées et sorties d'un programme algorithmique ;
- Connaître les variables, les opérateurs arithmétiques, logiques et de comparaison.

Introduction

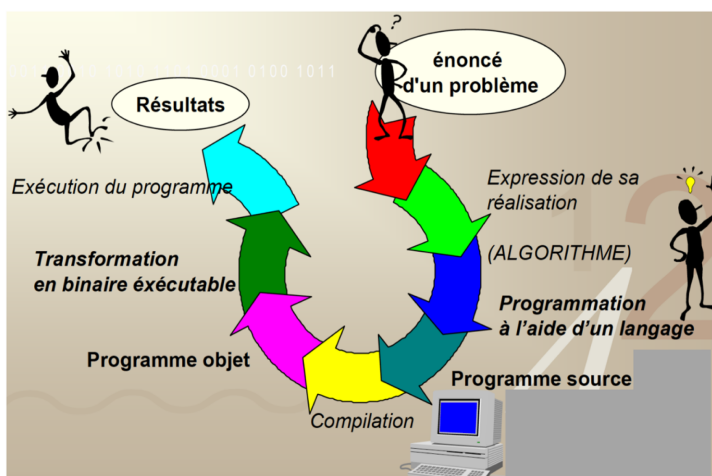


Le logiciel est l'un des constituants d'un ordinateur. Il est composé d'un ensemble de programmes qui résultent des solutions algorithmiques. Le mot algorithmique vient du mathématicien arabe du 9^{ème} siècle **Al-Khwarizmi**. Il reste la base dur de l'informatique et intervient dans le software et le hardware. Par ailleurs, le rôle d'une solution algorithmique est la résolution d'un problème précis. Une solution algorithmique est l'ensemble d'instructions définissant les opérations qui devront être exécutées de façon séquentielle par un ordinateur.

Étapes de résolution d'un problème (1/2)



Pour résoudre un problème en informatique, il faut passer par les étapes décrites à la figure ci - dessous.



- **Énoncé d'un problème** : Description d'un problème.
- **Algorithme** : Il est la phase d'analyse et de réflexion pour définir les données qu'on dispose et les objectifs qu'on souhaite atteindre. Il permet aussi de prévoir des réponses à tous les cas envisageables.
- **Programmation à l'aide langage** : traduction de l'algorithme en programme.

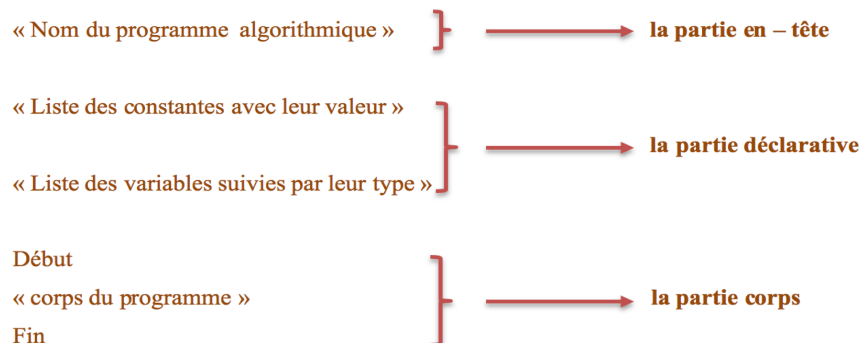
Étapes de résolution d'un problème (2/2)

- **Programme (ou code) source** : Il est généralement écrit dans un langage de programmation permettant d'être compris par les humains.
- **Compilation** : La compilation est opération réalisée par un programme. Le compilateur consiste à transformer le code source en langage machine (instructions et données).
- **Programme (ou code) objet** : Il représente une suite d'instructions en langage binaire (0 ou 1), uniquement compréhensible par l'ordinateur.
- **Transformation en binaire exécutable** : code binaire utilisé par le processeur pour exécuter le programme.
- **Exécution du programme** : Processus de mise en œuvre des instructions d'un programme pour obtenir un résultat.

Structure d'un programme algorithmique (1/1)

Le programme algorithmique comporte généralement trois parties :

- **La partie en - tête** : Elle porte le nom du programme algorithmique.
- **La partie déclarative** : Elle contient la déclaration des constantes, des types, des variables, etc.
- **La partie corps** : Elle est constituée de séquences d'actions faisant appel à des opérations de base d'un ordinateur. Une action peut être considérée comme une *affectation* ou une *entrée - sortie* ou un *contrôle conditionnel* ou une *répétition*.



Notion d'identificateurs et de types de données (1/1)

Dans un programme algorithmique, toutes les données doivent être identifiées (on parle d'identificateur). Pour un identificateur, nous avons :

- Le nom de l'identificateur doit être **alphabétique** ou **alphanumérique**.
- Un chiffre ne commence jamais un identificateur.
- Les composants d'un mot composé sont soit collés ou séparés par un tiret du **8** (`_`) (l'espace et le tiret du **6** (`-`) sont interdits).
- Il est recommandé à ce que le nom de l'identificateur ait en rapport avec son contenu.

Les types de données sont : **l'entier** (sans virgule), **le réel** (avec virgule), **le booléen** (vrai ou faux), **le caractère** (toujours entre apostrophe) et **la chaîne de caractère** (entre guillemets).

Notions de variable et de constante (1/2)

Une **variable** est un **emplacement mémoire** capable de contenir des **valeurs** de **type défini au préalable**. Une variable peut changer à l'intérieur d'un programme. Elle est composée par son **nom** (identificateur), sa **taille**, le contenu et son **adresse** (numéro du premier octet occupé par cette variable en mémoire). La syntaxe de la déclaration d'une variable est la suivante :

```
var identificateur : type;
```

Exemple :

```
var x, y : entier;  
var nom, adresse : chaîne;  
var A : réel;
```

Notions de variable et de constante (2/2)

La définition d'une **constante** est la même que celle d'une variable à la différence que la valeur d'une constante reste inchangée tout au long de l'algorithme. La syntaxe est la suivante :

```
const identificateur = valeur
```

Exemple :

```
const min = 20 ;  
const max = 300 ;  
const pi = 3.14 ;
```

Les instructions d'entrée/sortie (1/2)

L'instruction d'entrée ou de lecture permet d'affecter, à une variable, une donnée introduite à partir d'un périphérique d'entrée (clavier, fichier,...). La syntaxe est la suivante :

Lire(*nom_var1, nom_var2,...*) ou **Saisir**(*nom_var1, nom_var2,...*)

Par exemple **lire**(A) lit une valeur à partir du périphérique d'entrée et la range dans la case mémoire associée à A.

L'instruction de sortie ou d'écriture permet d'afficher des résultats sur un périphérique de sortie (écran). La syntaxe est la suivante :

Ecrire(*liste d'expression séparée par des virgules*) ou

Afficher(*liste d'expression*)

Les instructions d'entrée/sortie (2/2)

Exercice d'application :

Ecrire un programme algorithmique qui demande à l'utilisateur de saisir deux nombres au clavier.

Le programme détermine et affiche leur somme.

Solution algorithmique

Programme exercice

var X, Y : entier;

Début

Ecrire("Veuillez saisir la valeur de X");

Lire(X);

Ecrire("Veuillez saisir la valeur de Y");

Lire(Y);

Ecrire(X, " + ", Y, " = ", (X+Y));

Fin

Les opérateurs arithmétiques et logiques (1/4)

Pour effectuer une opération, nous devons associer des opérateurs et des opérands.

Une opération qui a un opérande et un signe est appelée **unaire**. Quant à l'opération binaire, elle est composée de deux opérands et un signe.

- **Les opérateurs arithmétiques**

L'informatique utilise les mêmes opérateurs qu'en mathématique.

Opérateurs	Exemples
Addition (+)	Unaire : +a ; +b. Binaire : a + b ; b + a.
Soustraction (-)	Unaire : -a ; -b. Binaire : a - b ; b - a.
Multiplication (*)	Opérateur non unaire, mais il est binaire. Binaire : a * b
Division réelle (/)	Opérateur non unaire, mais il est binaire. Binaire : a/b ; b/a.

Les opérateurs arithmétiques et logiques (2/4)



- Les opérateurs arithmétiques

Opérateurs	Exemples
Division entière (div)	Opérateur non unaire, mai il est binaire. Binaire : a div b
Reste de la division entière (mod)	Opérateur non unaire, mai il est binaire. Binaire : a mod b
Carré (sqr)	Opérateur non binaire, mai il est unaire. Binaire : sqr(a), sqr(b+a)
Racine carrée (sqrt)	Opérateur non binaire, mai il est unaire. Binaire : sqrt(a), sqrt(b+a)
Valeur absolue (abs)	Opérateur non binaire, mai il est unaire. Binaire : abs(a), abs(a+b)

NB : L'instruction d'affectation se note par ←

Exemple $x \leftarrow 12$. On attribue 12 à la variable x.

Les opérateurs arithmétiques et logiques (3/4)



- Les opérateurs logiques

Pour avoir une opération logique, on associe les variables et opérateurs logiques.

Une variable logique fonctionne comme une condition c'est - à - dire elle ne peut avoir que deux valeurs : Vrai (1) ou Faux (0).

Libellés	Opérateurs
ET (AND)	&&
OU (OR)	
NON (NOT)	!

Les opérateurs arithmétiques et logiques (4/4)



- **Les opérateurs de comparaison**

Les opérateurs de comparaison en informatique sont identiques à ceux en mathématique.

Les opérateurs sont donnés par le tableau suivant :

Opérateurs	Exemples
Inférieur (<)	$A < B$
Inférieur ou égal (≤)	$A \leq B$
Égalité (=)	$A = B$
Différent (< > ou ≠)	$A < > B$ ou $A \neq B$
Supérieur (>)	$A > B$
Supérieur ou égal (≥)	$A \geq B$

Exercice d'application



1. Écrire un algorithme qui lit deux entiers X et Y et affiche leur valeur avant et après permutation.
2. Écrire un algorithme qui lit trois entiers X, Y et Z et qui calcule et affiche leur produit et leur moyenne.
3. Écrire un algorithme qui permet de saisir une valeur entière puis affiche le carré de cette valeur.
4. Écrire un algorithme qui permet de saisir une expression de type chaîne de caractères puis affiche la valeur saisie.